Agentic LLM Systems:
Reverse and Forward Engineering

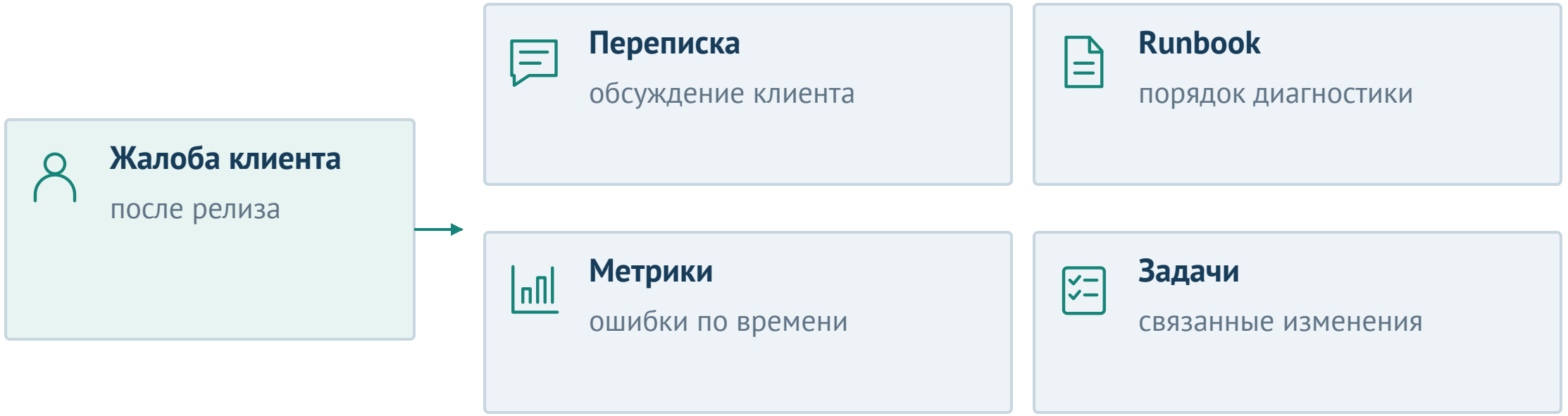
Лекция 6

Инструменты и среда агента

Алексеев Илья

DeepPavlov

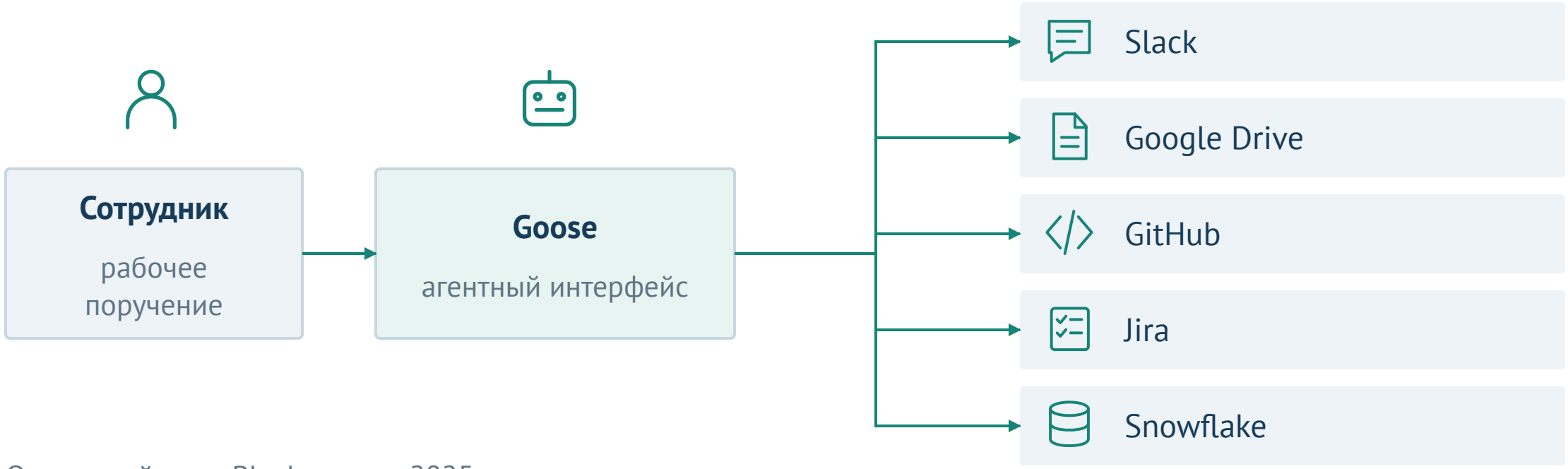
Рабочая задача сотрудника



Результат: задача разработчикам и черновик ответа с источниками

Один запрос требует данных и действий в нескольких системах.

Корпоративная среда Block



Описанный опыт Block, апрель 2025

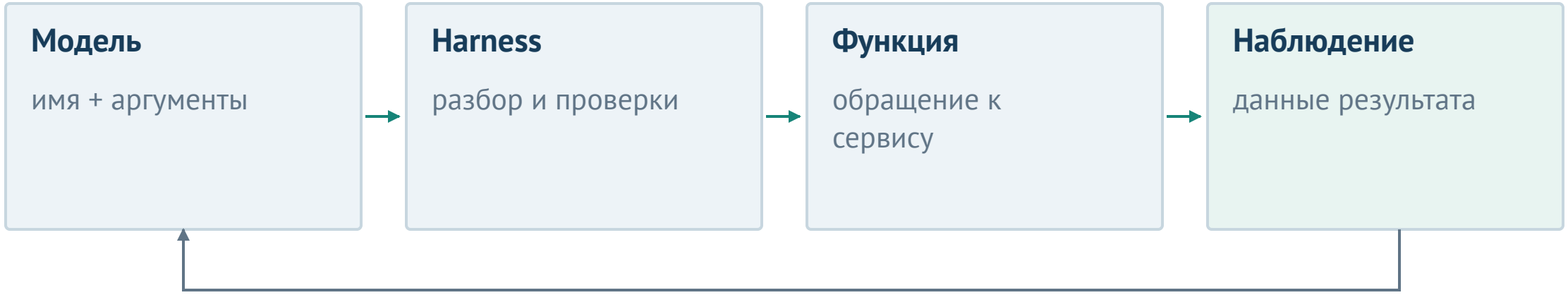
Harness связывает задачу сотрудника с подключёнными рабочими системами.

Источники: Block / Goose, 2025

Где исполняется инструмент

1. Локальная функция.
2. Подключение через MCP.
3. Sandbox и ресурсы исполнения.

Цикл вызова инструмента



Имя и аргументы проходят через harness, результат становится наблюдением следующего шага.

Источники: Pydantic AI: Tools

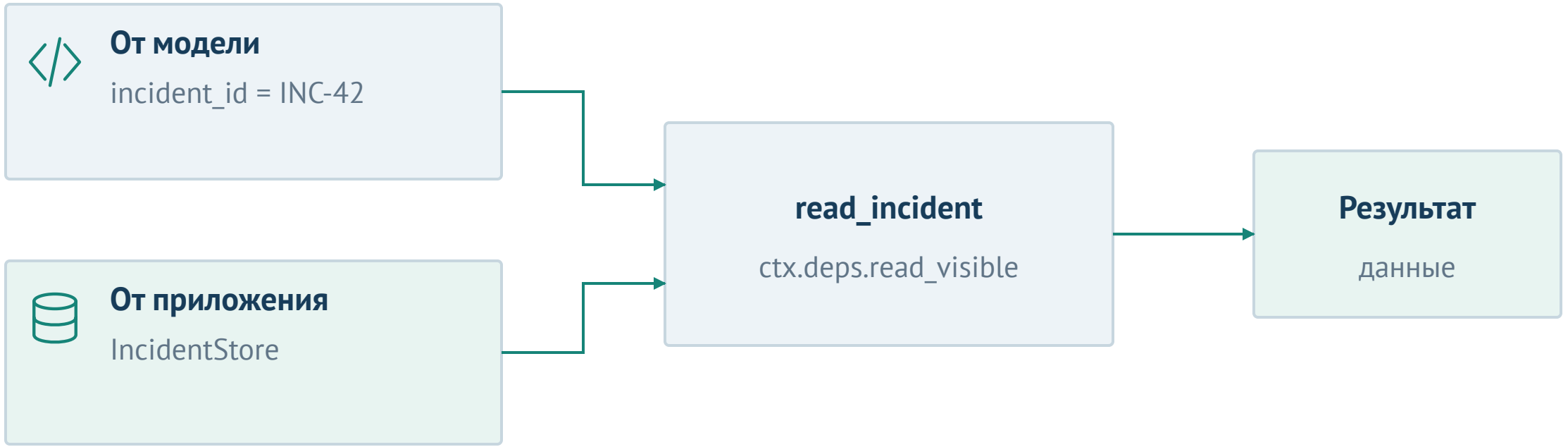
Локальный инструмент Pydantic AI

```python
@agent.tool
async def read_incident(
    ctx: RunContext[IncidentStore], incident_id: str
) -> dict:
    """Прочитать доступный сотруднику инцидент.

    Args:
        incident_id: Идентификатор, например INC-42.
    """
    return await ctx.deps.read_visible(incident_id)
```

Модель задаёт incident_id, приложение передаёт проверенную зависимость.

Аргументы и зависимости



ctx.deps не входит в схему аргументов модели

Модель задаёт аргументы, приложение передаёт зависимости и клиенты сервисов.

Built-in инструмент в коде приложения

Агентное приложение

```
@agent.tool
async def read_incident(
    ctx: RunContext,
    incident_id: str,
):
    return await (
        ctx.deps.read_visible(
            incident_id))
```

Плюсы

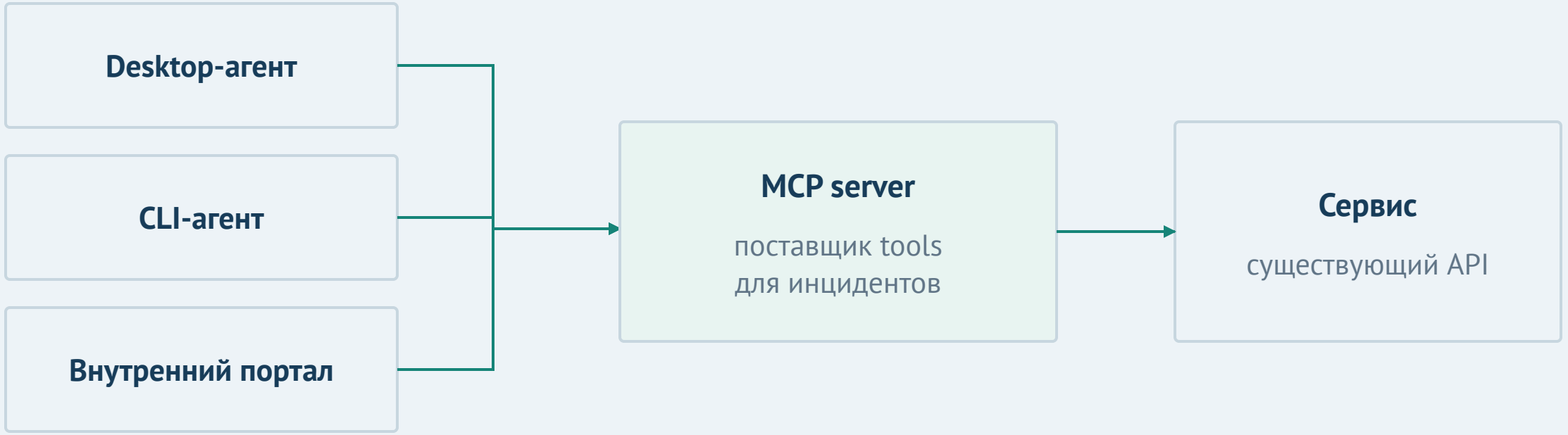
Прямой вызов
Обычные зависимости
Общая отладка

Ограничения

Поставка вместе с приложением
Другому приложению нужна адаптация

Built-in здесь: функция зарегистрирована и вызывается внутри агентного приложения.

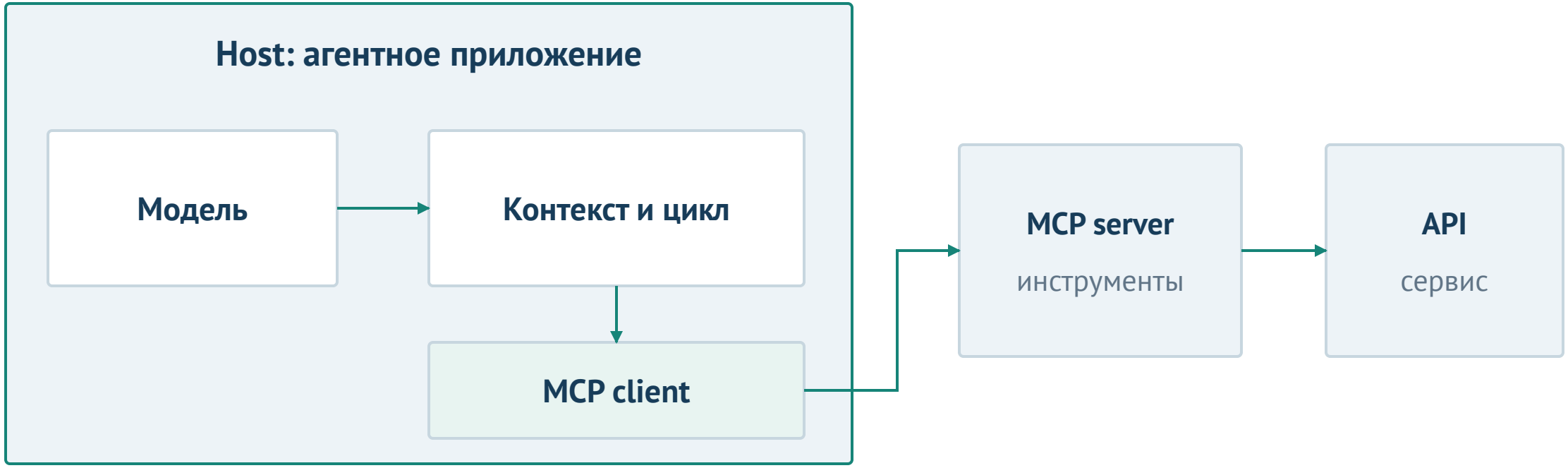
Одна интеграция для разных агентов



MCP (Model Context Protocol) задаёт общий контракт подключения инструментов к агентным приложениям.

Источники: MCP: Architecture

Host, client и server



Один client взаимодействует с одним server

Host — агентное приложение, client — его MCP компонент, server — поставщик инструментов.

Источники: MCP: Architecture

JSON-RPC: запрос и ответ

Запрос

```
{"jsonrpc":"2.0","id":7,
 "method":"subtract",
 "params":{"a":42,"b":23}}
```

method: процедура
params: входные данные

Ответ

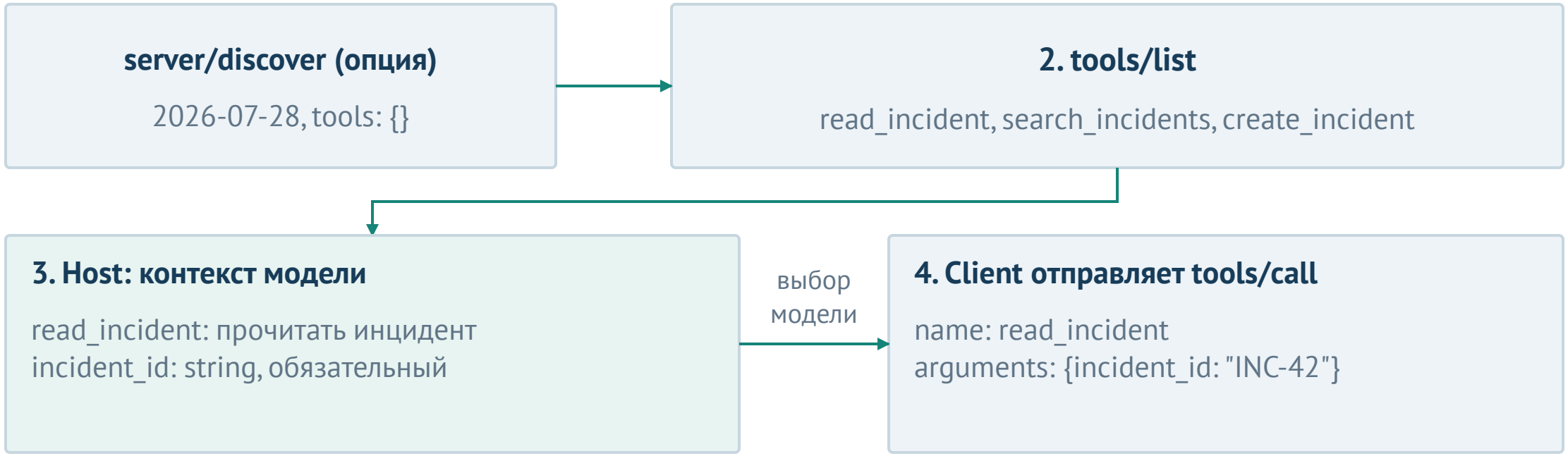
```
{"jsonrpc":"2.0","id":7,
 "result":19}
```

result: результат
error: ошибка вместо result

JSON-RPC описывает вызов процедуры сообщениями JSON; id связывает запрос с ответом.

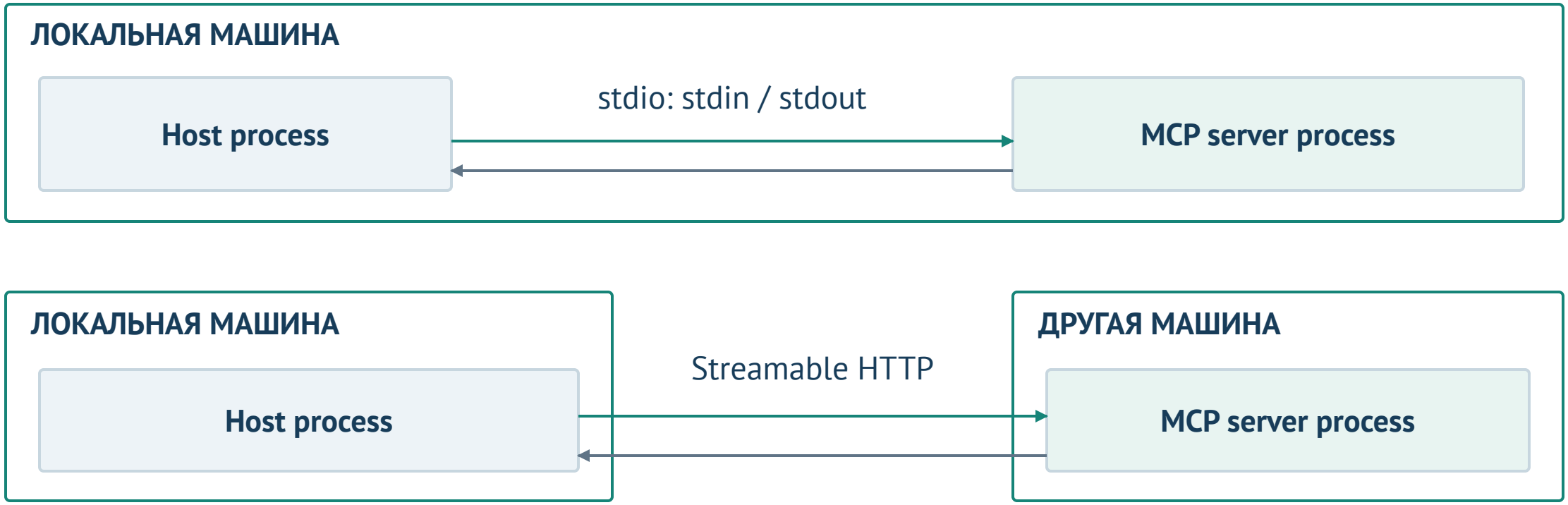
Каталог сервера и контекст модели

Настроен сервер: incidents.example/mcp



Сервер сообщает доступные tools; host выбирает определения для текущей задачи.

stdio и Streamable HTTP: размещение



stdio связывает локальные процессы; HTTP позволяет обратиться к серверу на другой машине.

Streamable HTTP: JSON или SSE

HTTP: запрос/ответ; Content-Type: формат тела; Accept: допустимые ответы

Вариант 1: обычный JSON ответ

```
HTTP/1.1 200 OK
Content-Type: application/json
```

пустая строка: начало тела HTTP ответа

```
{"jsonrpc":"2.0","id":7,"result":{"resultType":"complete","tools":[]}}
```

Вариант 2: тот же ответ в SSE событии

```
HTTP/1.1 200 OK
Content-Type: text/event-stream
```

пустая строка: начало тела HTTP ответа

```
data: {"jsonrpc":"2.0","id":7,"result":{"resultType":"complete","tools":[]}}
```

пустая строка: конец события

event: message необязателен: тип события по умолчанию message

SSE (Server-Sent Events) передаёт события в HTTP ответе; обычный JSON ответ тоже допустим.

SSE 1/3: ход выполнения tools/call

Независимый ответ 1/3: tools/call id=1; _meta.progressToken="job-1"

```
HTTP/1.1 200 OK
Content-Type: text/event-stream
```

пустая строка: начало тела HTTP ответа

```
data: {"jsonrpc":"2.0","method":"notifications/progress","params":{"progressToken":
↪"job-1","progress":10,"total":100,"message":"Обработано 10 файлов"}}
```

пустая строка: конец события

```
data: {"jsonrpc":"2.0","method":"notifications/progress","params":{"progressToken":
↪"job-1","progress":70,"total":100,"message":"Обработано 70 файлов"}}
```

пустая строка: конец события

↪ только экранный перенос; в конспекте каждый data: занимает одну физическую строку

progressToken="job-1" связывает уведомления о ходе; у них method без id.

SSE 1/3: результат завершает вызов

Ответ 1, продолжение: после двух progress событий

```
  data: {"jsonrpc":"2.0","id":1,"result":{"resultType":"complete","content":[{"type":
↪"text","text":"Готово. Обработано 100 файлов, ошибок нет."}]}}
```

пустая строка: конец события

id=1: результат именно исходного tools/call

progressToken="job-1": только связь уведомлений о ходе

После результата server закрывает поток (SHOULD в спецификации).

↪ только экранный перенос; в конспекте каждый data: занимает одну физическую строку

Ответ с id=1 завершает tools/call; progressToken не заменяет JSON-RPC id.

SSE 2/3: сообщения logging

Независимый ответ 2/3: id=2; logging: {}; logLevel="info" (фрагменты)

```
HTTP/1.1 200 OK
Content-Type: text/event-stream
```

пустая строка: начало тела HTTP ответа

```
data: {"jsonrpc":"2.0","method":"notifications/message","params":{"level":"info",
↪"logger":"report","data":{"message":"Начинаю сбор данных"}}}
```

пустая строка: конец события

```
data: {"jsonrpc":"2.0","method":"notifications/message","params":{"level":"warning",
↪"logger":"report","data":{"message":"Один источник недоступен, использую кэш"}}}
```

пустая строка: конец события

↪ только экранный перенос; в конспекте каждый data: занимает одну физическую строку

Logging deprecated в 2026-07-28; пример требует capability logging и logLevel="info".

SSE 2/3: итог отдельно от журнала

Ответ 2, продолжение: финальный result с id=2

```
  data: {"jsonrpc":"2.0","id":2,"result":{"resultType":"complete","content":[{"type":
↪"text","text":"Отчёт готов. Для одного источника использованы данные из кэша."}]}}
```

пустая строка: конец события

Журнал: params.data.message в notifications/message

Результат инструмента: result.content в response id=2

Поток закрывается после результата. Logging deprecated.

↪ только экранный перенос; в конспекте каждый data: занимает одну физическую строку

data.message описывает ход исполнения; result.content содержит результат tools/call id=2.

SSE 3/3: длительная подписка

Resource: данные MCP; URI: идентификатор, например file:///config.json.

Подписка: будущие изменения. Независимый ответ 3/3: listen id=3.

```
HTTP/1.1 200 OK
Content-Type: text/event-stream
```

пустая строка: начало тела HTTP ответа

```
data: {"jsonrpc":"2.0","method":"notifications/subscriptions/acknowledged","params":
↪{"_meta":{"io.modelcontextprotocol/subscriptionId":3},"notifications":{
↪"toolsListChanged":true,"resourceSubscriptions":["file:///config.json"]}}}
```

пустая строка: конец события

↪ только экранный перенос; в конспекте каждый data: занимает одну физическую строку

subscriptions/listen id=3 открывает третий ответ; первое событие подтверждает согласованный фильтр.

SSE 3/3: изменения приходят позже

Ответ 3, продолжение: события после acknowledged

```
  data: {"jsonrpc":"2.0","method":"notifications/resources/updated","params":{"_meta":
↪{"io.modelcontextprotocol/subscriptionId":3},"uri":"file:///config.json"}}
```

пустая строка: конец события

```
  data: {"jsonrpc":"2.0","method":"notifications/tools/list_changed","params":{
↪"_meta":{"io.modelcontextprotocol/subscriptionId":3}}}
```

пустая строка: конец события

Новое содержимое: отдельный resources/read по URI. Поток открыт.

↪ только экранный перенос; в конспекте каждый data: занимает одну физическую строку

subscriptionId=3 связывает уведомления с listen id=3; ответ остаётся открыт для новых изменений.

MCP и модель ISO/OSI

- **7 Прикладной**
- 6 Представления
- 5 Сеансовый
- 4 Транспортный
- 3 Сетевой
- 2 Канальный
- 1 Физический

L7: прикладные протоколы и форматы

HTTP: перенос запроса и ответа

Ответ: JSON либо SSE с JSON сообщениями

JSON-RPC: конверт вызова

MCP: методы tools/list, tools/call и их смысл

stdio: локальный IPC

Межпроцессный обмен вне сетевой OSI схемы

MCP, JSON-RPC, SSE и HTTP относятся к L7; их вложенность описывает разные задачи внутри L7.

Источники: ITU-T X.200: OSI model · JSON-RPC 2.0: Specification · MCP: Transports · MCP 2026-07-28: Streamable HTTP

Когда built-in, когда MCP

Критерий	Built-in функция	MCP интеграция
Потребитель	Одно агентное приложение	Разные hosts
Поставка	Вместе с приложением	Независимо от host
Обслуживание	Код и зависимости приложения	Протокол, процесс или сервис

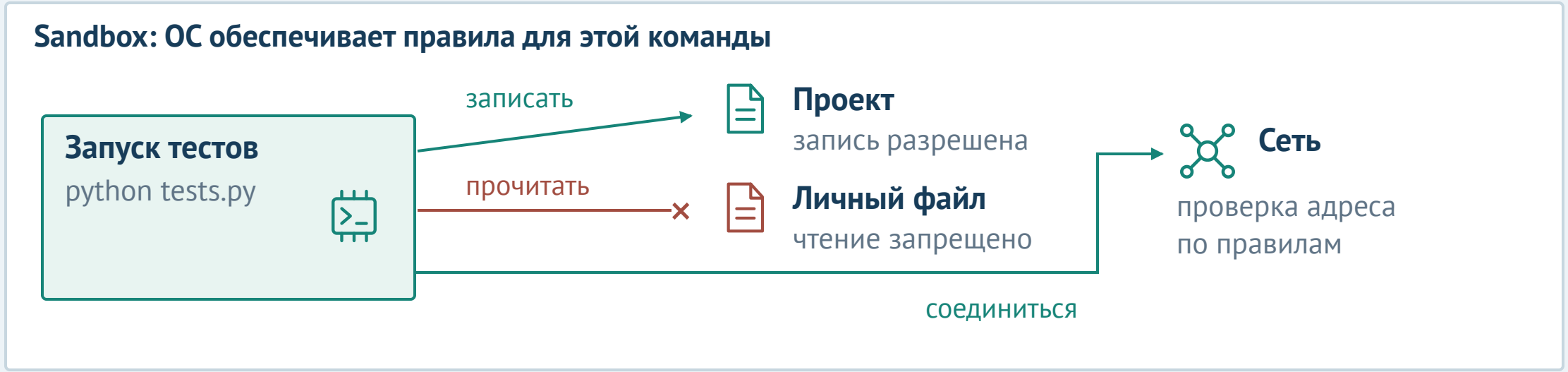
Built-in удобен внутри одного приложения; MCP полезен для независимого подключения к разным hosts.

Источники: Pydantic AI: Tools · MCP: Architecture

Sandbox и ресурсы команды

Workload — исполняемая задача: команда и её дочерние процессы

Политика — правила доступа. Ниже учебный пример, а не настройки по умолчанию



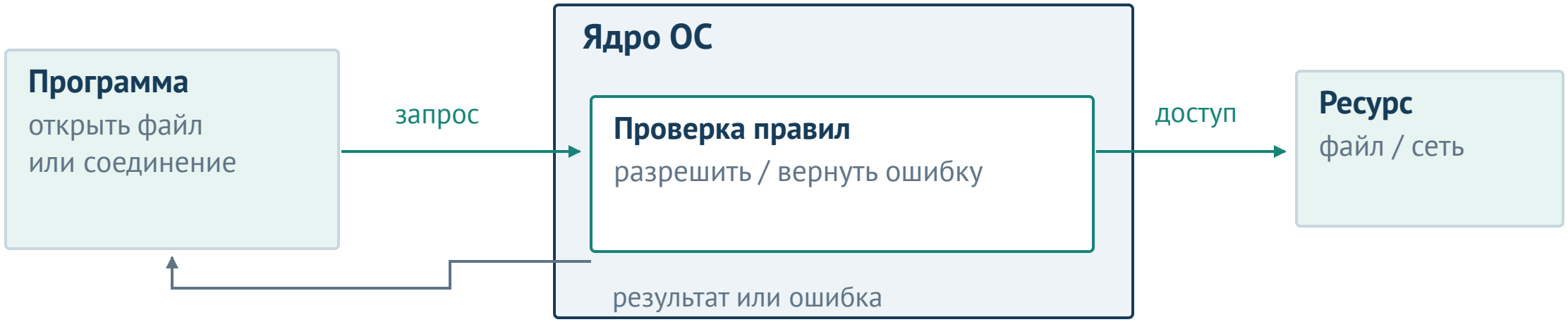
Sandbox ограничивает ресурсы исполняемой задачи по заданным правилам.

Источники: Codex: Approvals & security · Claude Code: Sandboxing

Программа, системный вызов и ядро

Системный вызов — запрос программы к ядру ОС

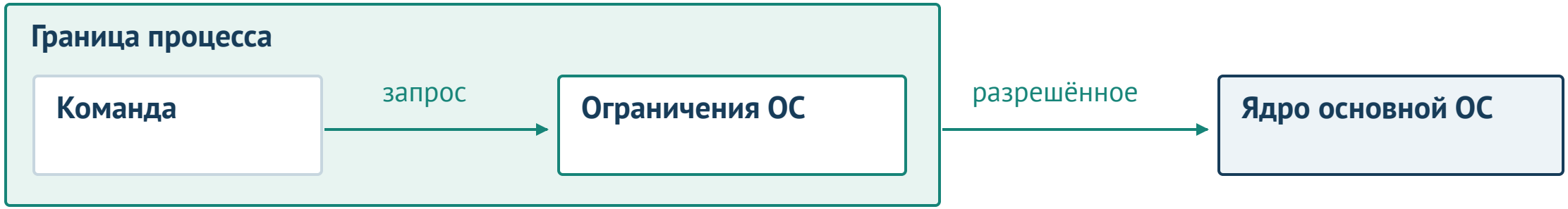
Ядро (kernel) управляет процессами, памятью и доступом к ресурсам



Правила исполняет ОС; текст «не читай этот файл» сам не создаёт границу

Ядро обслуживает запросы программ к файлам, сети и другим ресурсам.

Ограничение обычного процесса



Seatbelt · macOS
Профиль разрешённых операций процесса

bubblewrap · Linux
Утилита: отдельный вид файлов и других ресурсов

seccomp · Linux
Фильтр системных вызовов; часть общей политики

У процесса остаётся общее ядро с остальными программами на машине

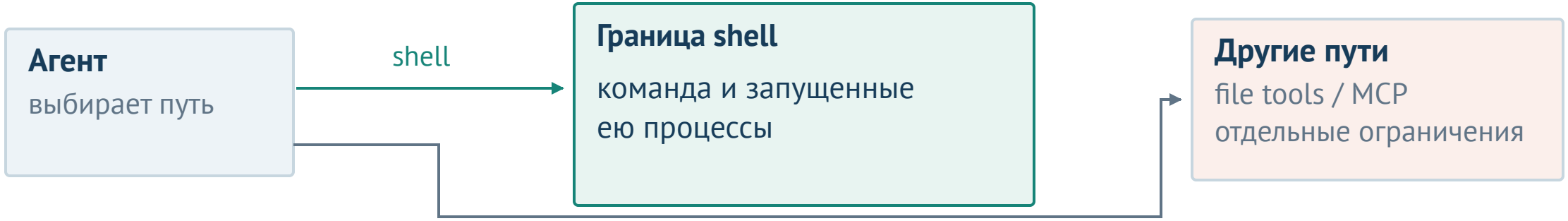
ОС применяет ограничения к процессу, работающему на этой же машине.

Источники: Codex: Approvals & security · bubblewrap: Sandbox construction · Linux: Seccomp filters

Shell sandbox в Codex и Claude Code

Локальные команды	macOS	Linux
Codex	Seatbelt	bubblewrap + seccomp
Claude Code	Seatbelt	bubblewrap

Область действия shell sandbox



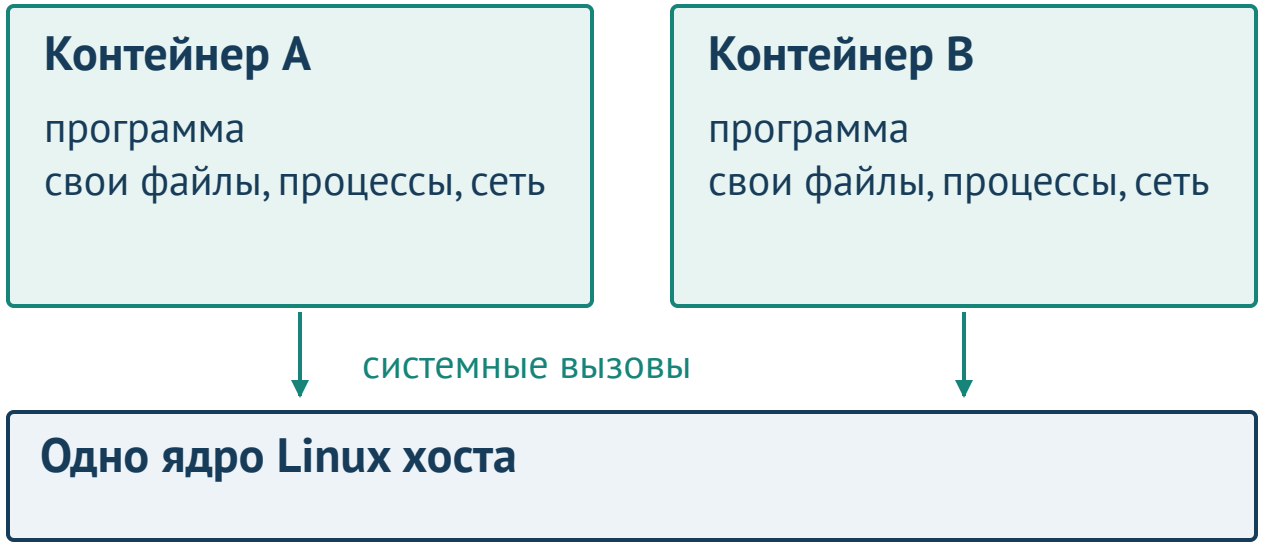
8 ноября 2026 · Доступные команде ключи, токены и переменные среды не скрываются автоматически

Граница команды и её потомков охватывает только конкретный путь исполнения.

Источники: Codex: Approvals & security · Claude Code: Sandboxing · Codex: Permissions

Контейнер и общее ядро

Контейнер — отдельное окружение процессов с выбранными файлами и зависимостями



namespaces

Отдельные представления файлов, процессов и сети

cgroups

Учёт и лимиты ресурсов: память, время процессора

Docker создаёт такие среды. Хост — машина, на которой они запущены

Контейнер отделяет окружение процессов, сохраняя общее ядро Linux.

Источники: Docker: Security

gVisor: посредник вместо прямого вызова

gVisor — среда изоляции с собственной реализацией системного интерфейса ОС



Мотивация: сократить прямое взаимодействие чужого кода с ядром общей машины

Sentry реализует системные вызовы приложения в отдельном программном слое.

Источники: gVisor: Security model

Виртуальная машина и microVM



Виртуальная машина запускает собственное гостевое ядро на виртуальном оборудовании.

Источники: Firecracker: Design · Linux: KVM API

Подключение и исполнение

- **Протокол** задаёт обмен и контракт возможностей.
- **Размещение** определяет, где выполняется код.
- **Sandbox** ограничивает ресурсы конкретного исполнения.

Как сервис становится инструментом

1. Данные и документы.
2. Предметные действия и браузер.
3. Делегирование агентам.

ClickHouse: анализ событий

Сотрудник: «После какой версии выросло число ошибок?»

SQL-консоль · учебный запрос

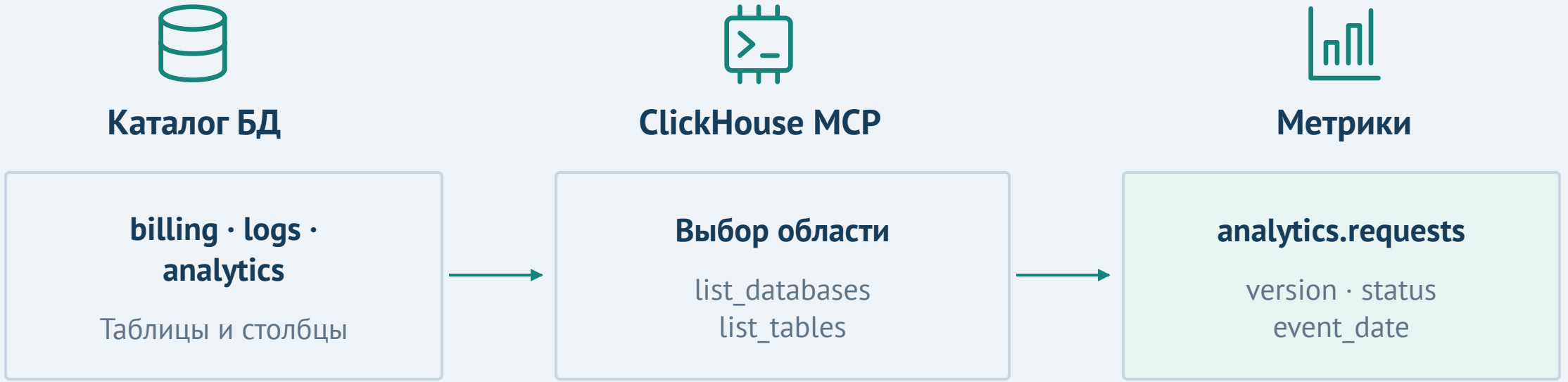
```
SELECT version, count() AS errors
FROM analytics.requests
WHERE status >= 500 AND event_date = '2026-11-07'
GROUP BY version
```

version	errors
v1	12
v2	87

SQL задаёт, какие строки отобрать и какую сводку вычислить

Аналитическая БД хранит события и вычисляет сводки по большим таблицам.

Какая таблица отвечает на вопрос

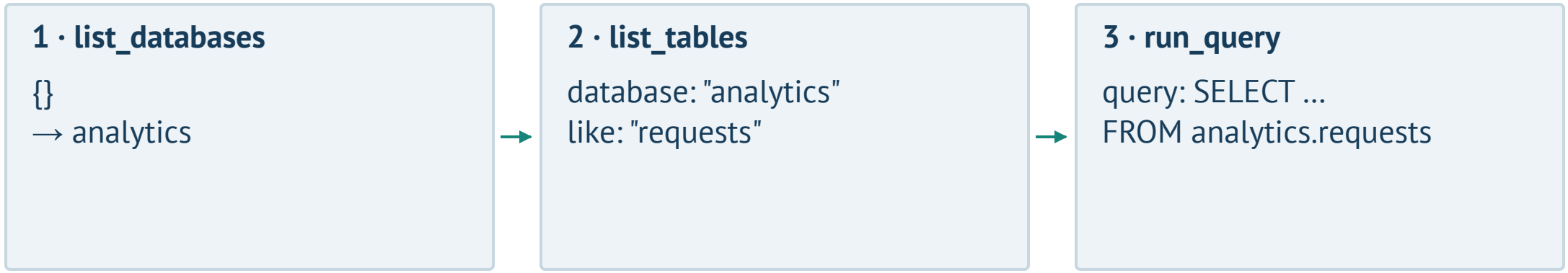


Агенту нужна структура выбранных данных перед составлением SQL

Адаптер раскрывает структуру БД для выбора данных и SQL-запроса.

Источники: ClickHouse MCP

Метаданные перед запросом



Если список неполон: next_page_token → следующий page_token

Структуру данных можно раскрывать по мере уточнения задачи.

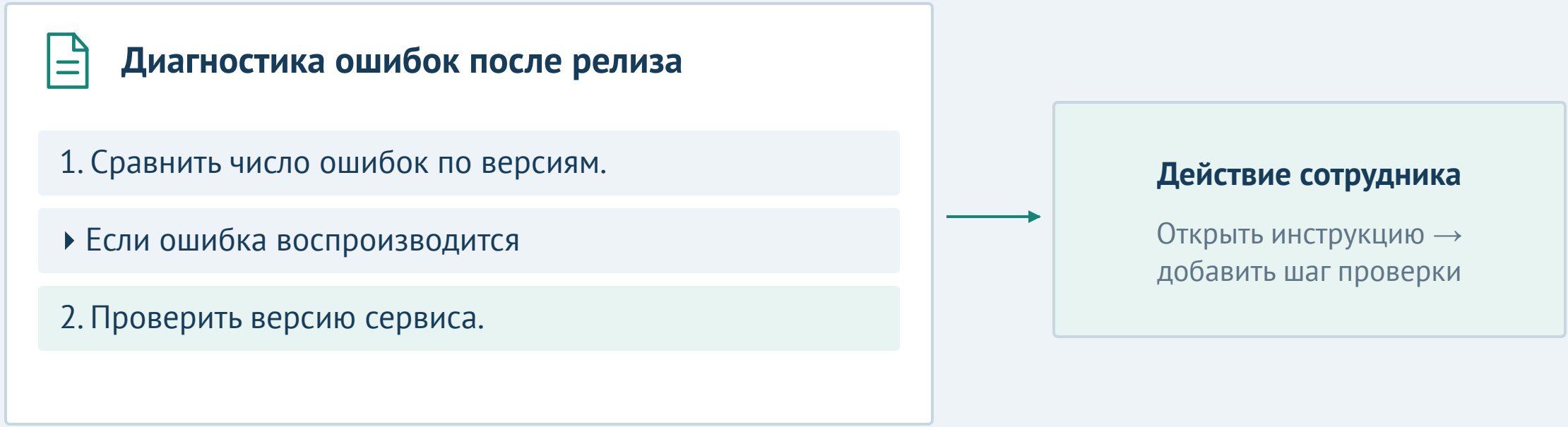
Ограничения ответа и выполнения

Проблема	Решение адаптера	Граница
Много таблиц	Страницы list_tables	Не лимит SQL-результата
SQL не выполнен	Структурированная ошибка	Исправить запрос или вход
Долгое ожидание	Timeout и попытка отмены	Остановка не гарантирована

Пагинация метаданных, timeout и права БД решают разные задачи.

Источники: ClickHouse MCP: Changelog

Notion: общие документы команды

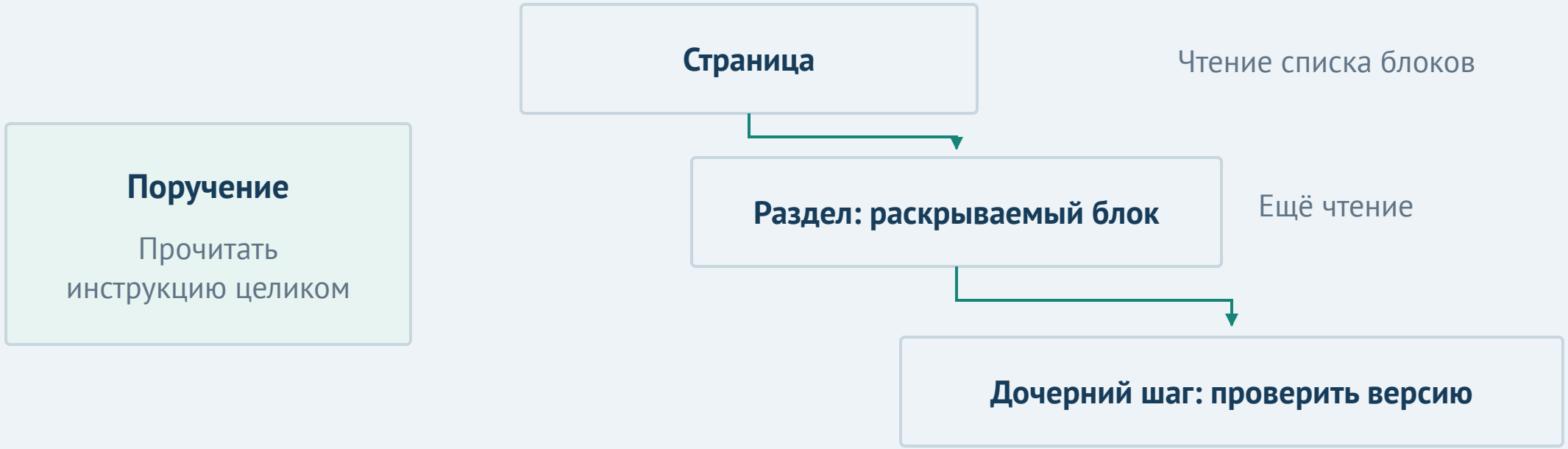


Каждая строка или вложенная часть документа хранится как блок

Страница — документ из блоков: заголовков, абзацев, списков и вложенных частей.

Источники: Notion: Pages and blocks

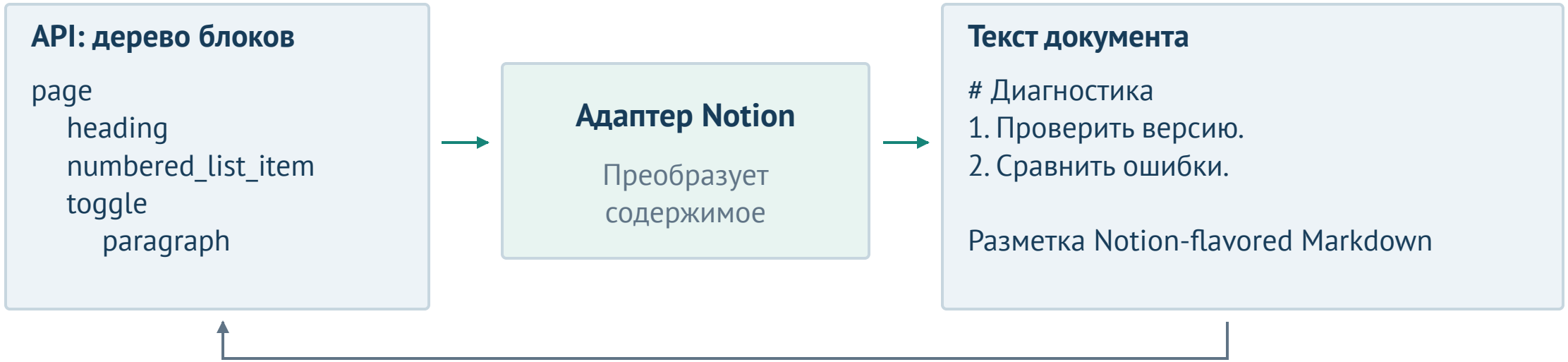
Почему дерево блоков мешает чтению



Прямое отображение endpoint → tool сохраняет служебные шаги обхода

Первая версия адаптера переносила в контекст техническую структуру документа.

От блоков к тексту документа



При записи адаптер преобразует разметку в представление сервиса

Notion-flavored Markdown представляет содержимое в форме, удобной для чтения и правки.

Явная неполнота документа

notion-fetch: часть инструкции

```
# Диагностика
1. Проверить версию.
[раздел не загружен]

truncated: true
unknown_block_ids: ["block-A"]
```



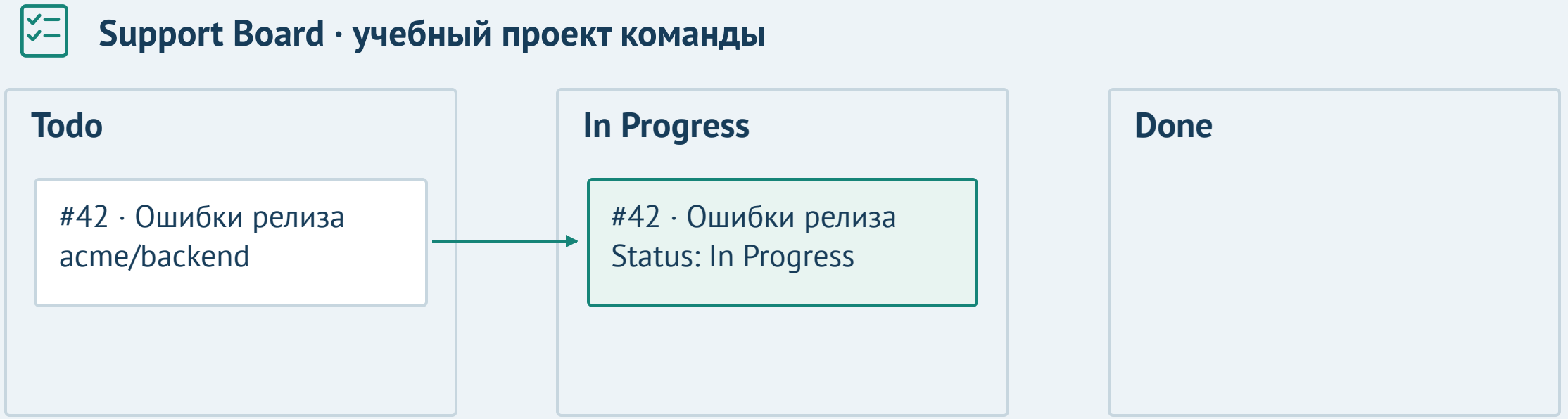
Дочитать конкретный раздел

Повторить notion-fetch
с возвращённым ID block-A

Учебный ID · если доступ запрещён, повтор не открывает содержимое

Признак усечения и адрес фрагмента позволяют дочитать нужную часть.

GitHub Projects: план работы команды



Сотрудник переносит карточку; у записи проекта меняется поле Status

Issue хранит описание задачи; Project показывает задачи в таблице или на доске.

Источники: GitHub: About Projects

Задача, запись, поле и значение

Понятие	Что означает	Учебный пример
Project	План работы / доска	Support Board, № 7
Issue	Задача в репозитории	acme/backend #42
Item	Запись задачи в проекте	Связь #42 с доской № 7
Field	Поле записи проекта	Status
Option	Вариант поля с выбором	In Progress

У одной задачи и её записи в проекте разные идентификаторы.

Источники: GitHub: About Projects · GitHub MCP: projects.go

Перевести issue #42 в In Progress

Задача: у issue #42 на Support Board установить Status = In Progress

Проект уже найден: owner = acme, project_number = 7

1 · Найти запись

acme/backend #42
→ item в проекте № 7

→

2 · Найти значение

Status → field
In Progress → option

→

3 · Изменить поле

Записать option ID
в поле найденного item

Агент задаёт предметные ссылки; сервер разрешает внутренние ID

Адаптер связывает задачу, запись проекта и вариант поля Status.

Источники: GitHub MCP: projects.go

Три tools, несколько операций

REST — HTTP-запрос к адресу ресурса; GET — чтение, PATCH — изменение.

GraphQL — запрос/изменение через именованные поля. org — организация.

Tool / аргумент method	Операция сервиса (владелец — org)
projects_list · list_projects	REST: GET /orgs/{owner}/projectsV2
projects_get · get_project	REST: GET /orgs/{owner}/projectsV2/{number}
projects_write · update_project_item	GraphQL: разрешить ссылки → REST: PATCH item
projects_write · create_project_status_update	GraphQL: createProjectV2StatusUpdate

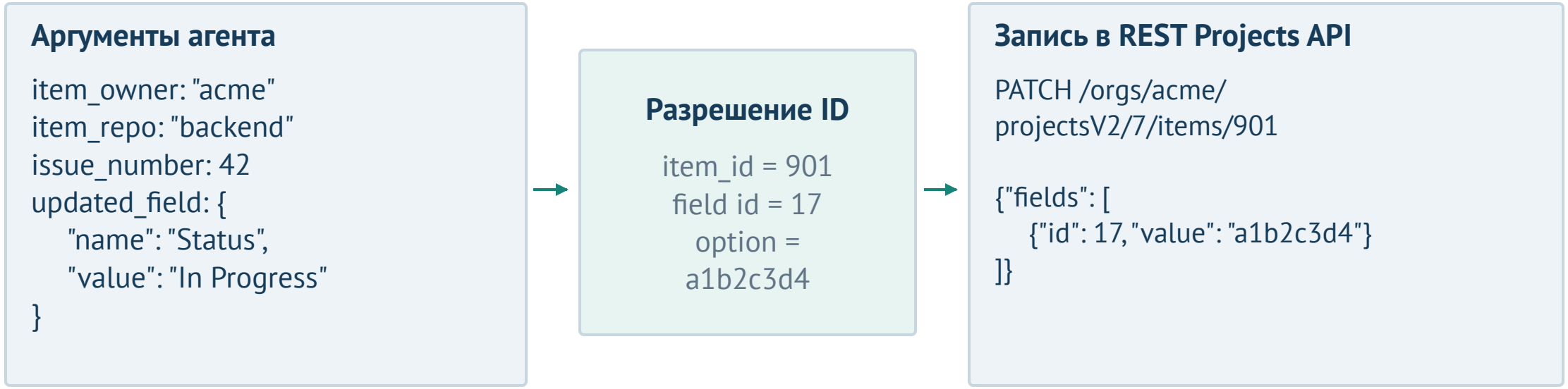
Имя tool выбирает семейство; аргумент method — конкретную операцию.

Один полный вызов projects_write

```
{"jsonrpc": "2.0", "id": 1, "method": "tools/call",
  "params": {
    "_meta": {
      "io.modelcontextprotocol/protocolVersion": "2026-07-28",
      "io.modelcontextprotocol/clientCapabilities": {}
    },
    "name": "projects_write", "arguments": {
      "method": "update_project_item",
      "owner": "acme", "project_number": 7,
      "item_owner": "acme", "item_repo": "backend", "issue_number": 42,
      "updated_field": {"name": "Status", "value": "In Progress"}
    }}
}
```

tools/call — метод MCP; update_project_item — аргумент инструмента.

До и после разрешения имён

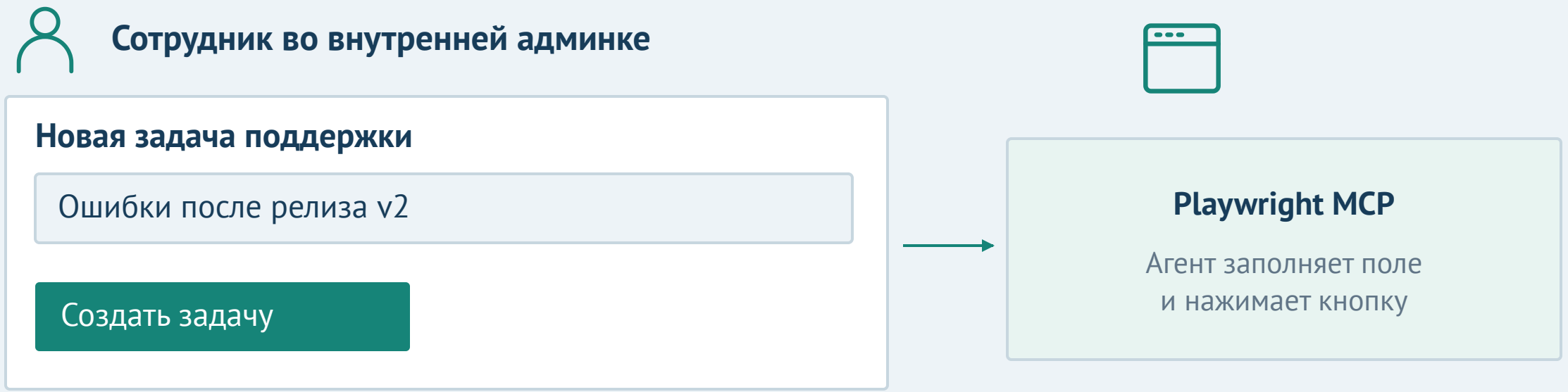


ID здесь учебные; при работе сервер получает их от API GitHub.

Исключение: create_project требует owner_type явно (user или org).

Сервер получает действительные ID из GitHub и подставляет их в запись.

Playwright: действия в браузере



Автоматизируется маршрут в браузере: открыть → заполнить → нажать

Автоматизация браузера программно открывает страницы, заполняет поля и нажимает кнопки.

Страница как наблюдение для агента

Страница админки · момент t

Новая задача
Заголовок: Ошибки после релиза

[Создать задачу]

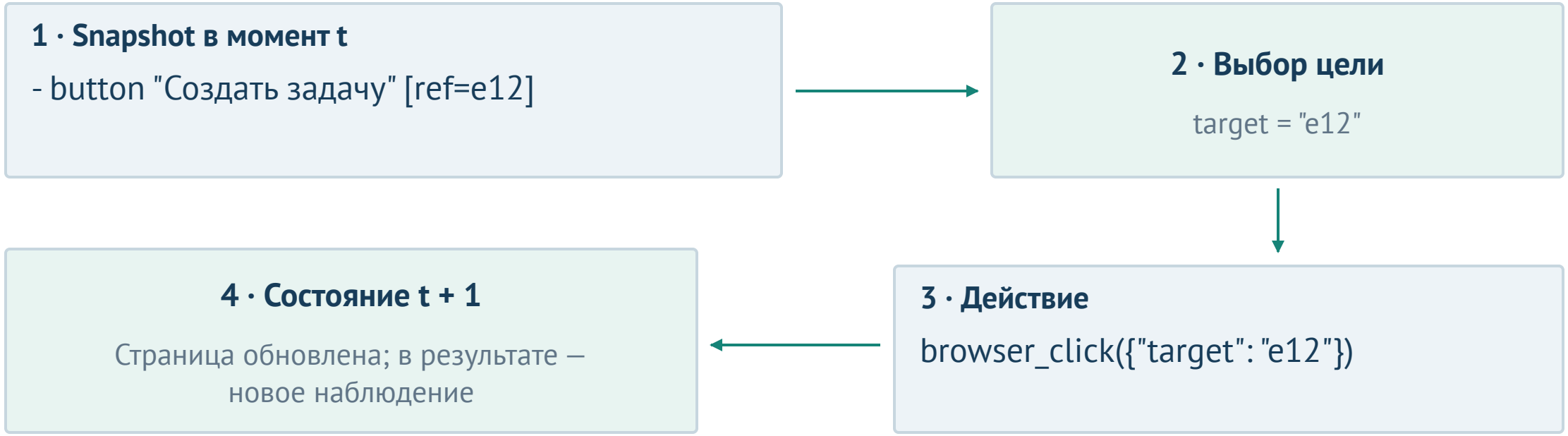
Accessibility snapshot · то же состояние

```
- heading "Новая задача"
- textbox "Заголовок" [ref=e8]
    value: Ошибки после релиза
- button "Создать задачу" [ref=e12]
```

Роль: button · имя: «Создать задачу» · ссылка для действия: e12

Accessibility snapshot описывает роли, названия и ссылки элементов страницы.

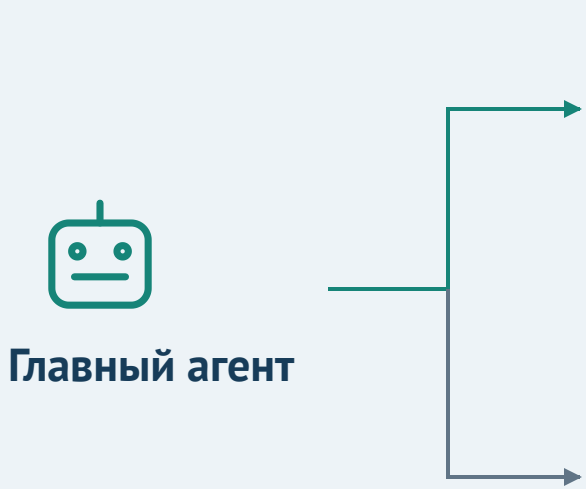
Наблюдение → цель → действие



Следующий target выбирают по актуальному состоянию страницы

Цель берётся из состояния t; после клика агент наблюдает состояние t + 1.

Поручение агенту аналитики



Поручение агенту аналитики

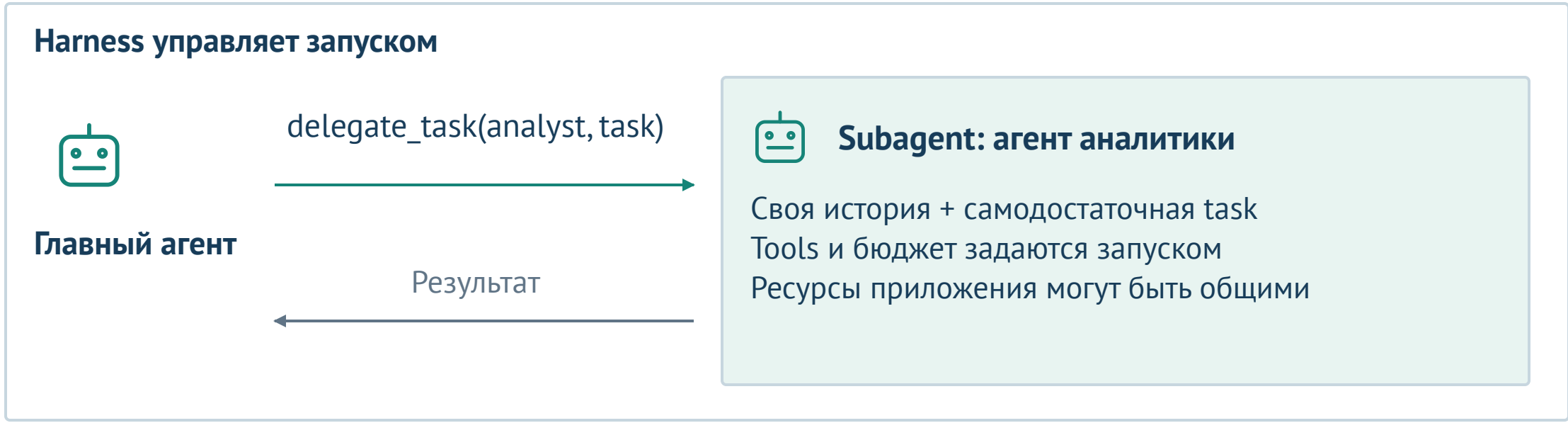
Почему после r42 выросли ошибки?
Период: 30 минут до и после релиза.
Результат: выводы + подтверждающая таблица.

Фиксированный SQL-tool

Одна заданная выборка → строки результата

Tool задаёт операцию; поручение задаёт результат для собственного цикла исполнителя.

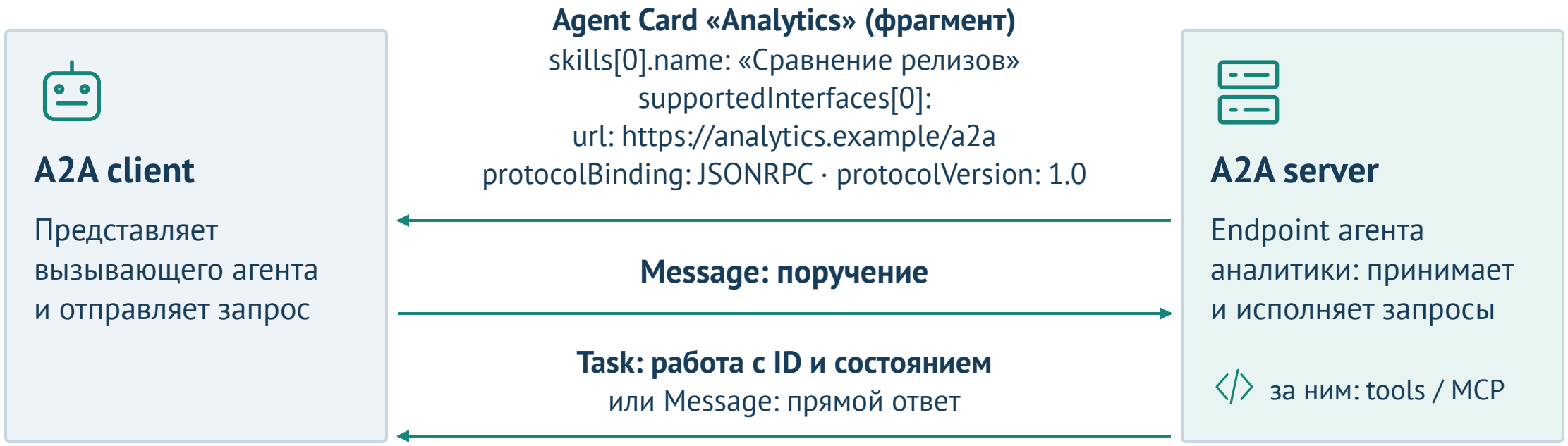
Subagent: контракт запуска в harness



Отдельная история не изолирует общий workspace

Subagent — дочерний агентный запуск; fork означает выбранное наследование контекста.

A2A: вызывающая и исполняющая стороны



Client/server — роли обмена; одна система может выполнять обе

Client отправляет поручение; server публикует Agent Card и выполняет задачи.

Поручение → задача → артефакт

Client → SendMessage

Message m-7
Сравни ошибки r41/r42
за 30 минут; верни CSV.

Учебное поручение

Server → Task t-17

state: TASK_STATE_COMPLETED

Artifact a-1: errors.csv

r41: 1% ошибок
r42: 4% ошибок

Сокращённые объекты A2A 1.0.0; возможен и прямой ответ Message

Task хранит состояние работы; Artifact содержит полученный результат.

Как передаются A2A-сообщения

Binding	Вызовы и данные	Поток обновлений
JSON-RPC	Методы + JSON по HTTP(S)	SSE: события по HTTP
HTTP+JSON/REST	HTTP(S): JSON + ресурсные URL	SSE: события по HTTP
gRPC	Типизированные вызовы: Protobuf по HTTP/2 + TLS	gRPC server stream



Webhook: HTTP POST на заданный URL с обновлением задачи

Опциональные push notifications; принимающий endpoint настраивается отдельно

Python SDK v1.3.0: три binding; WebSocket / Kafka — отдельный адаптер

Binding связывает операции A2A с форматом запросов и сетевым протоколом.

Решения создателей адаптеров

- Данные раскрываются в нужной области и представлении.
- Предметная операция включает служебную работу адаптера.
- Следующий шаг следует из результата и состояния среды.

Корпоративная среда агента

1. Контракт инструмента и общий каталог.
2. Авторизация между приложением и сервисами.
3. Контекст, уточнение, длительная работа и интерфейс.

Описание помогает выбрать операцию

Задача: «Отметь INC-42 как решённый»

```
def set_incident_status(
 incident_id: str,
 status: Literal["investigating", "resolved"]
) -> IncidentState:
 """Изменить состояние существующего инцидента.
 incident_id: ID записи, например INC-42.
 status: investigating — расследование идёт;
 resolved — работа восстановлена.
 Возвращает ID, состояние и ссылку source.
 Не создаёт инцидент и не уведомляет клиента."""
```

Docstring

Документационная строка → description инструмента.

Предметные термины

Инцидент — запись о сбое.
INC-42 — ID этой записи.

Модель выбирает операцию по имени и описанию; проверки выполняет сервис.

Docstring объясняет назначение, аргументы и границы действия.

Схема задаёт вход и выход

inputSchema → arguments

```
{"type":"object",
 "properties":{
 "incident_id":{"type":"string"},
 "status":{"type":"string",
 "enum":["investigating","resolved"]}
 },
 "required":["incident_id","status"],
 "additionalProperties":false}
```

IncidentState → structuredContent

```
{"incident_id":"INC-42",
 "status":"resolved",
 "source":
 "https://issues.example/INC-42"}
```

type — тип; required — обязательность.
enum — допустимый словарь.
outputSchema — схема результата.

Форма JSON проверяема автоматически; допустимость перехода — правило сервиса.

JSON Schema ограничивает форму данных; права и переход состояния проверяет сервис.

Отказ операции — часть контракта

Протокольная ошибка

Неизвестный инструмент: вызов не найден.

```
{"jsonrpc":"2.0","id":9,
 "error":{"code":-32602,
 "message":"Unknown tool"}}
```

Ошибка выполнения

Инструмент найден: переход запрещён.

```
{"jsonrpc":"2.0","id":9,
 "result":{"resultType":"complete",
 "isError":true,"content":[{
 "type":"text",
 "text":"INC-42 is already closed"
 }]}}
```

Исправить имя операции ≠ повторить запрещённый переход.

Ошибка должна сообщать причину и допустимое продолжение.

Каталог делает интеграции обнаружимыми

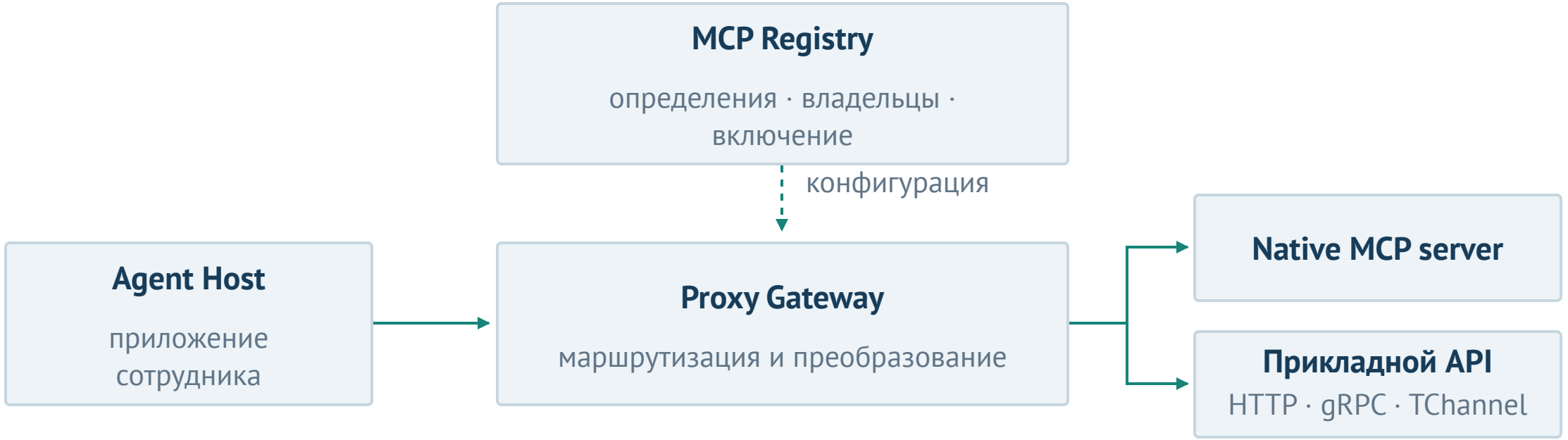
Интеграция	Владелец	Пример инструмента	Подключение
Инциденты	SRE	set_incident_status	включено
Документация	Knowledge team	search_runbooks	включено
Метрики	Observability	query_metrics	проверка

Обнаружение контракта и доступ к объекту — отдельные решения.

У каждой возможности есть владелец, определение и состояние подключения.

Источники: Uber: MCP Gateway, 2026

Registry и gateway: архитектура Uber



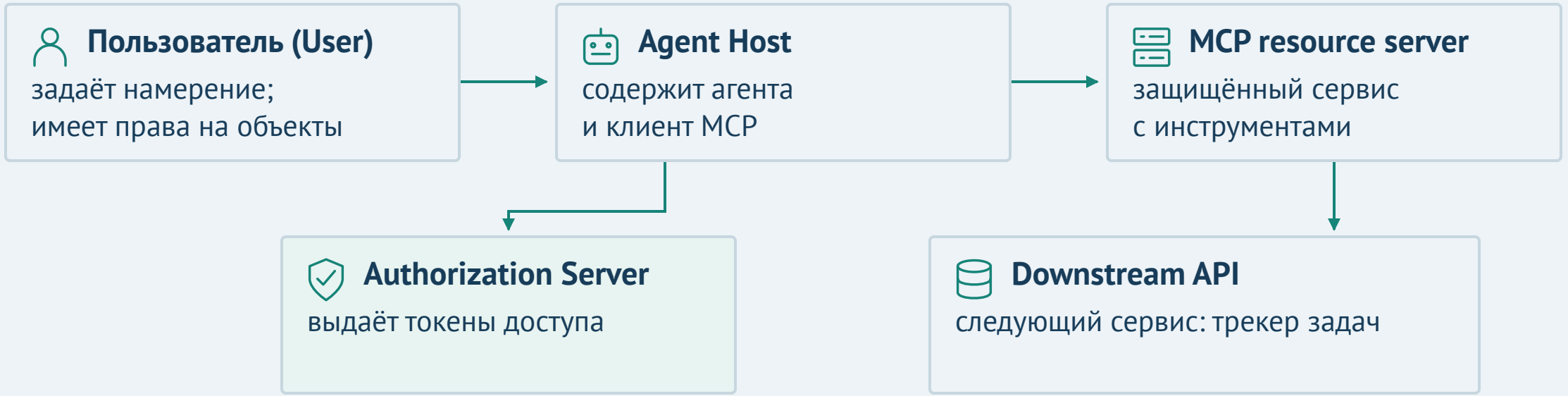
Registry управляет определениями; gateway обслуживает конкретные вызовы.

Registry хранит определения; Proxy Gateway маршрутизирует вызовы.

Источники: Uber: MCP Gateway, 2026

Кто действует и кто проверяет доступ

Identity — удостоверенная идентичность; principal — субъект проверки доступа.



Субъект доступа — пользователь или приложение, чьи права проверяются.

Как клиент получает доступ к MCP

Access token

Выданное свидетельство разрешённого доступа.

Audience

Сервис, которому предназначен токен.

Scope

Именованная область операций: incidents:write.

1. Найти адреса

401: нужен токен WWW-Authenticate → metadata ресурса → metadata OAuth

→

2. Согласовать

User → OAuth server: вход и consent на доступ приложению

→

3. Получить

Host → OAuth server: код + PKCE + resource → access token A

→

4. Предъявить

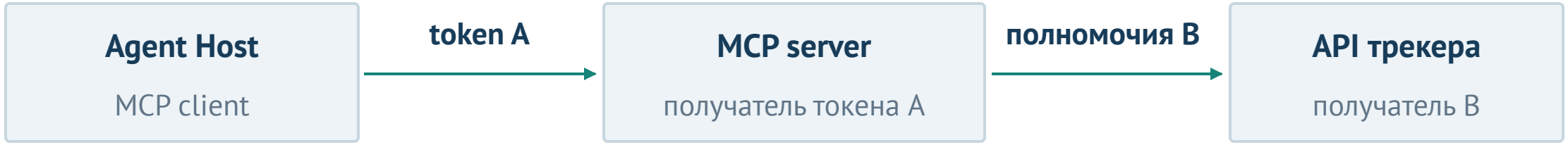
Host → MCP server: Authorization: Bearer <token A>

consent — согласие; PKCE — защита обмена кода; resource — URI MCP server.

Discovery находит сервер авторизации; OAuth выдаёт токен для MCP resource server.

Источники: MCP: Authorization

Два сервиса — две границы доступа



Проверка в каждом сервисе

Токен валиден? Верны audience и scopes?
Субъект может менять именно INC-42?

А нельзя переслать как B

Для следующего API нужны полномочия, предназначенные именно этому API.

Delegated: B связан с правами пользователя.

App-only: B использует собственную учётную запись приложения.

MCP не требует единого механизма обмена токенов для всех downstream API.

MCP server проверяет свой токен и объект; downstream API — свои полномочия.

Согласование, авторизация и sandbox

Проверка	Конкретный вопрос	Кто применяет
Согласование	«Одобряю закрытие INC-42»?	User + Agent Host
Авторизация	Может ли этот субъект изменить INC-42?	MCP server + API
Sandbox	Разрешены ли процессу эти файлы и соединения?	Среда исполнения

Одобрение человека не выдаёт серверные права и не меняет sandbox.

Каждая проверка отвечает на свой вопрос; одна не заменяет другую.

Что ещё может предоставить MCP

Потребность	Возможность	Статус на 08.11.2026
Получить контекст	Resources / Prompts	Core
Уточнить проект	Elicitation	Core: 2026-07-28
Дождаться аналитики	Tasks	Beta extension
Исследовать результат	Apps	Stable extension

Дополнительные возможности требуют явной поддержки со стороны host.

Каждая возможность решает отдельную задачу взаимодействия.

Источники: MCP: Extensions · MCP: Elicitation

Resources и prompts: контекст для задачи

Resource — данные по URI

Runbook: инструкция восстановления сервиса.

```
resources/read
{"uri":"runbook://checkout/recovery"}
→ contents: [{uri, mimeType, text}]
```

Приложение добавляет выбранные данные в контекст.

Prompt — шаблон сообщений

Разбор инцидента: роли, текст и параметры.

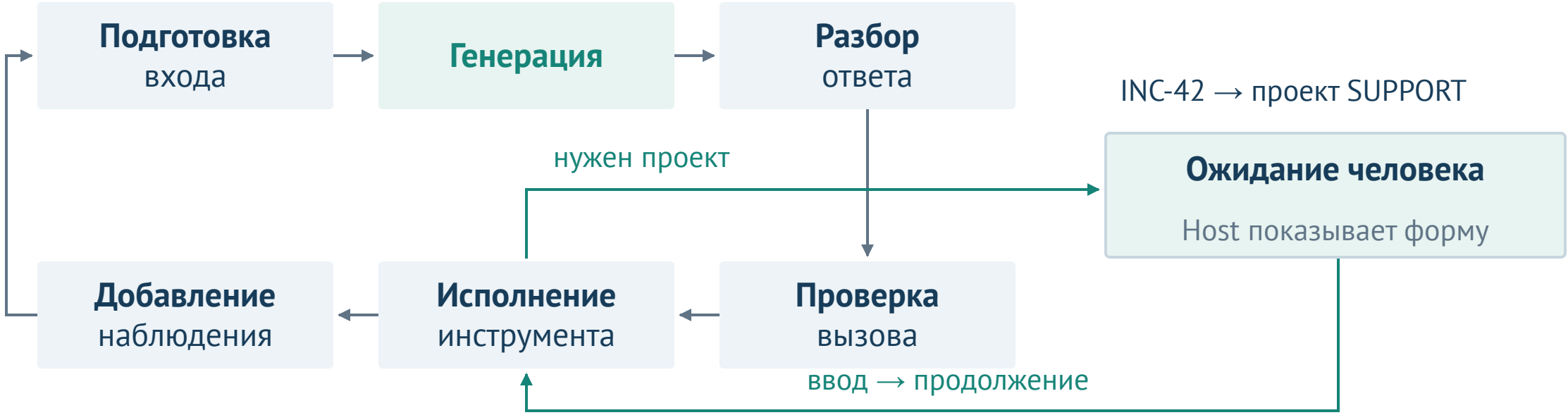
```
prompts/get
{"name":"incident_review",
 "arguments":{"incident_id":"INC-42"}}
→ messages: [{role, content}]
```

Пользователь выбирает сценарий подготовки сообщений.

Resource предоставляет содержимое; prompt — готовую структуру сообщений.

Elicitation в знакомом агентном цикле

Elicitation — запрос недостающих сведений через host. Цикл из лекции 3.



Во время выполнения host ожидает уточнение и затем продолжает тот же запрос.

Elicitation: исходный HTTP-запрос

```
POST /mcp HTTP/1.1
Host: mcp.example
Content-Type: application/json
Accept: application/json, text/event-stream
MCP-Protocol-Version: 2026-07-28
Mcp-Method: tools/call
Mcp-Name: prepare_issue
{"jsonrpc":"2.0","id":71,"method":"tools/call","params":{
 "name":"prepare_issue","arguments":{"incident_id":"INC-42"},
 "_meta":{
 "io.modelcontextprotocol/protocolVersion":"2026-07-28",
 "io.modelcontextprotocol/clientInfo":{"name":"helpdesk","version":"1"},
 "io.modelcontextprotocol/clientCapabilities":{"elicitation":{"form":{}}}
 }}}
```

В каждом запросе клиент сообщает версию MCP и поддержку формы.

Elicitation: сервер просит выбрать проект

HTTP 200 · Content-Type: application/json · ответ на id 71


```
{"jsonrpc":"2.0","id":71,"result":{
 "resultType":"input_required","requestState":"opaque-state-1",
 "inputRequests":{"project":{"method":"elicitation/create","params":{
 "mode":"form","message":"Выберите проект для задачи",
 "requestedSchema":{"type":"object","properties":{
 "project":{"type":"string","enum":["SUPPORT","PLATFORM"]}
 },"required":["project"]}
 }}}}}
```


Form — поля формы без паролей, API keys и access tokens.

URL — внешнее взаимодействие: host показывает домен и запрашивает согласие.

InputRequiredResult содержит запрос формы и непрозрачное состояние продолжения.

Elicitation: ввод и продолжение

Новый POST /mcp с теми же HTTP headers; JSON-RPC id меняется с 71 на 72

```
{"jsonrpc":"2.0","id":72,"method":"tools/call","params":{
 "name":"prepare_issue","arguments":{"incident_id":"INC-42"},
 "inputResponses":{"project":{
 "action":"accept","content":{"project":"SUPPORT"}}},
 "requestState":"opaque-state-1","_meta":{
 "io.modelcontextprotocol/protocolVersion":"2026-07-28",
 "io.modelcontextprotocol/clientInfo":{"name":"helpdesk","version":"1"},
 "io.modelcontextprotocol/clientCapabilities":{"elicitation":{"form":{}}}
 }}}
```

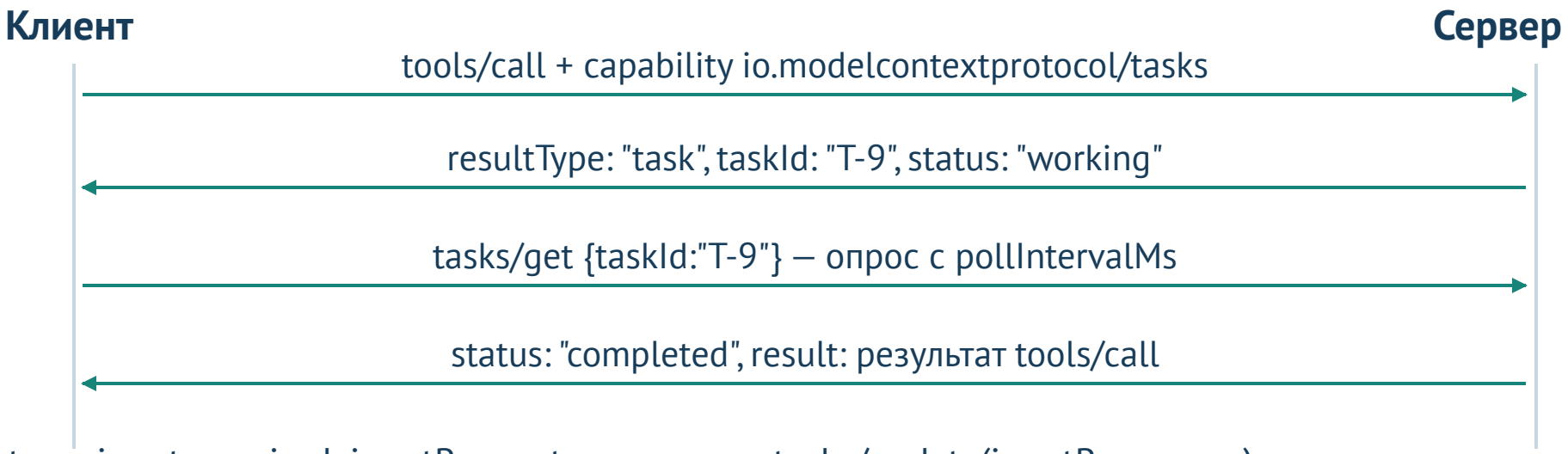
MRTR (Multi Round-Trip Requests): несколько обменов для одной операции.

Новый JSON-RPC id; те же arguments; inputResponses и requestState переданы в params.

Источники: MCP: Elicitation · MCP 2026-07-28: MRTR · MCP 2026-07-28: Schema

Tasks: результат можно получить позже

Tasks · Beta · payloads 2026-07-28



Если status = input_required: inputRequests → ввод → tasks/update(inputResponses).

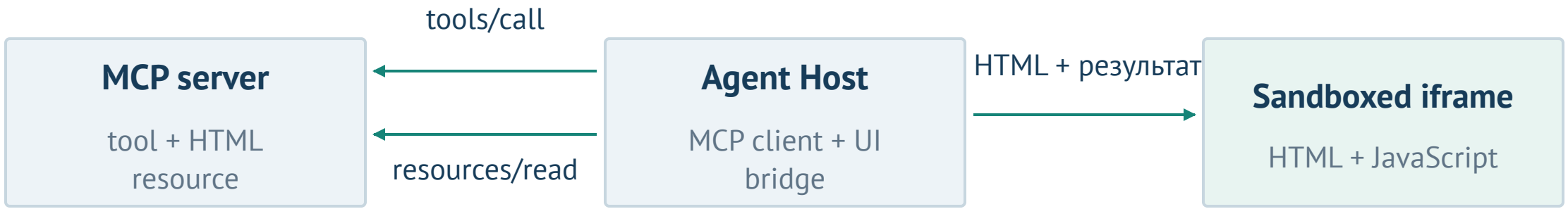
taskId — дескриптор операции; сервер проверяет доступ. tasks/cancel — запрос отмены.

taskId связывает запуск, опрос состояния и получение конечного результата.

MCP Apps: инструмент и UI-ресурс

Apps: payloads 2026-01-26 · поверх MCP · ресурс text/html;profile=mcp-app

```
tools/list → name: "show_incident_metrics",
 _meta: {"ui":{"resourceUri":"ui://metrics/view","visibility":["model","app"]}}
```



Модель: текст content. Host: UI metadata и загрузка HTML по ui://.

Мост (bridge) передаёт сообщения между host и изолированным представлением.

Host читает HTML по ui:// и связывает изолированный интерфейс с результатом инструмента.

MCP Apps: сообщения проходят через host

Apps 2026-01-26: ui/initialize → ui/notifications/initialized → обмен сообщениями

App → Host · JSON-RPC через postMessage

```
{"jsonrpc":"2.0","id":8,"method":"tools/call",
 "params":{"name":"query_metrics","arguments":{"incident_id":"INC-42"}}}
```

Host → App · ответ на запрос приложения, тот же id:8

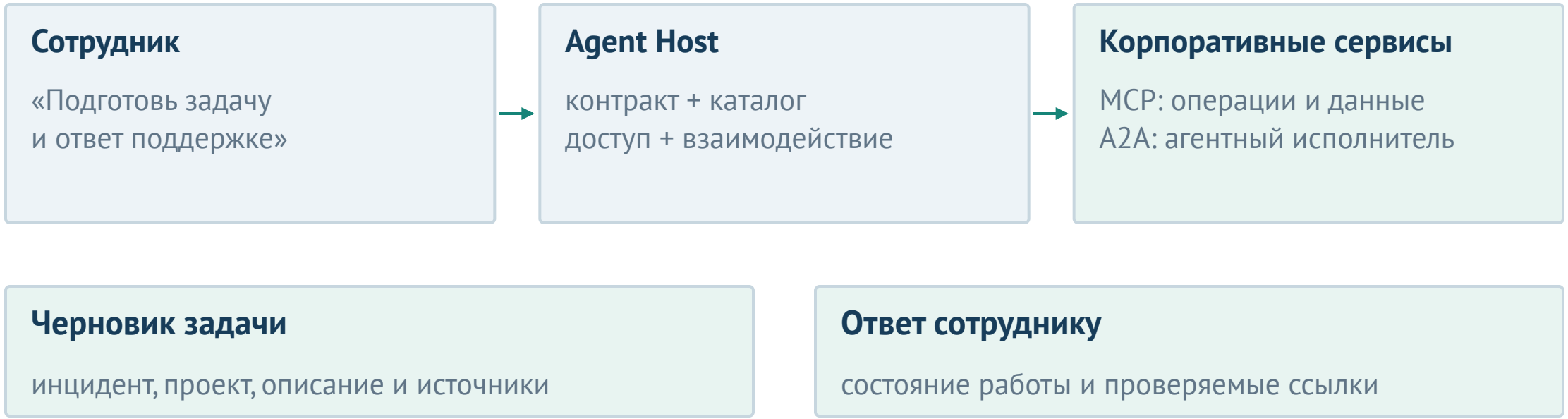
```
{"jsonrpc":"2.0","id":8,"result":{
 "content":[{"type":"text","text":"Ошибки после релиза выросли"}],
 "_meta":{"series":[2,7,11]}}}
```

Host проверяет источник и visibility; API — права. _meta передаётся UI, не модели.

JSON-RPC через postMessage связывает UI с host; сервер по-прежнему проверяет доступ.

Источники: MCP Apps: payloads 2026-01-26

Рабочая среда сотрудника



Понятный контракт, корректный доступ и подходящая форма результата связывают системы в рабочий процесс.